

Lab 09 · El control de acceso

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

90 minutos · Spring Boot 4.1.0 · Java 25 (Temurin) · PostgreSQL 16 embebido

Resumen

La diferencia entre «no te conozco» y «te conozco, pero esto no es para ti»: **401 sin token, 403 con token y sin el rol**. Y por qué la misma clave guardada dos veces produce dos hashes distintos.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	2
1.4. La puesta a punto	3
2. El caso	3
2.1. El control de acceso, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · La puerta abierta	3
3.2. Paso 2 · Una línea, y todo se cierra	4
3.3. Paso 3 · Las claves no se guardan: se codifican	5
3.4. Paso 4 · El carnet: emitir un token	6
3.5. Paso 5 · 401 no es 403	8
4. Lo que aprendiste	10
5. Para profundizar	11
6. Antes de cerrar	11

1 Antes de empezar

1.1 Qué vas a lograr

Todos los endpoints que has escrito hasta ahora **los puede llamar cualquiera**. Hoy se cierra la puerta.

Vas a poner un guardia en la entrada, vas a ver por qué una contraseña **nunca** se guarda tal cual —y por qué la misma clave produce dos hashes distintos—, vas a emitir un carnet que el portador lleva encima, y vas a distinguir dos respuestas que casi todo el mundo confunde: el **401** y el **403**.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs 01 a 04 hechos	Sabes crear endpoints y usar un repositorio	Imprescindible
Estar en la carpeta del lab	cd labs/lab-09-seguridad/practica	El cd no da error
curl a mano	Vas a mandar cabeceras	El navegador no basta hoy

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. Con Java no importa. El código completo está en labs/lab-09-seguridad/solucion/.

1.4 La puesta a punto

```
cd labs/lab-09-seguridad/practica
./mvnw spring-boot:run
```

Escucha en el **8095** y su PostgreSQL en el **55440**. **Párala con Ctrl+C**.

2 El caso

La oficina de la DGT tiene un mostrador de consultas abierto a todo el mundo, y **una sala interna** donde están los datos de fiscalización. Hasta hoy, cualquiera podía entrar a las dos.

2.1 El control de acceso, que es la metáfora de este laboratorio

Un guardia en la puerta y un carnet.

Cuando la oficina crece, se pone **un guardia**. El guardia hace dos preguntas distintas, y confundirlas es el error del día:

1. «**¿Quién es usted?**» — eso es **autenticación**. Si no puedes demostrarlo, el guardia no te deja pasar de la entrada. Es el **401**: *no te conozco*.
2. «**¿Puede usted entrar ahí?**» — eso es **autorización**. Ya sabe quién eres; lo que pasa es que **esa sala no es para tu categoría**. Es el **403**: *te conozco, y no*.

Para no tener que identificarte en cada puerta, la oficina te da **un carnet** al entrar. El carnet lleva escrito tu nombre y tu categoría, y **va firmado** con un sello que sólo la DGT sabe hacer. Cualquier puerta puede comprobar la firma sin llamar a recepción. Eso es el token.

Y las contraseñas: la oficina **no guarda tu clave**. Guarda algo que le permite comprobar que la sabes, sin poder deducirla. Como una huella: sirve para verificar y no se puede convertir de vuelta en un dedo.

3 Los pasos

3.1 Paso 1 · La puerta abierta

Qué vamos a hacer

Comprobar, antes de tocar nada, que hoy entra cualquiera.

Para entenderlo mejor

Entrar a la sala interna sin que nadie pregunte nada. Es el punto de partida, y conviene verlo.

Se corre

```
curl -i localhost:8095/productos
```

Lo que vas a ver

La respuesta llega, con su contenido. **Sin identificarse, sin nada**.

Vas bien si...

La API responde 200 a cualquiera. Es lo que vas a cerrar hoy.

Si te atascas

1 · EL PUERTO 55440 YA ESTA OCUPADO O ESTE MISMO PROYECTO YA ESTA CORRIENDO

Los dos candados del Lab 04, con los números de este lab:

```
lsof -ti:55440 | xargs kill -9
```

3.2 Paso 2 · Una línea, y todo se cierra

Qué vamos a hacer

Añadir la dependencia de seguridad y ver qué pasa **sin configurar nada**.

Para entenderlo mejor

Contratar al guardia. Todavía no le has dado instrucciones, así que aplica la instrucción por defecto: **no pasa nadie**.

El problema

Una aplicación sin seguridad no es «una aplicación a la que le falta una capa»: es una aplicación **abierta**. Y lo peligroso de eso es que funciona perfectamente, así que nadie lo nota.

La alternativa, y por qué no

Se podría dejar todo abierto por defecto y que cada endpoint pidiera protección. Spring Security hace lo contrario a propósito: **cierra todo y tú abres lo que quieras**. Un endpoint nuevo nace protegido, y olvidarse de protegerlo deja de ser posible — sólo puedes olvidarte de **abrirlo**, que se nota enseguida.

Se pega

En practica/pom.xml, **dentro de <dependencies>**:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Lo que vas a ver

Reinicia y vuelve a pedir cualquier cosa: **401**. Todo cerrado, sin haber escrito una línea de configuración.

Vas bien si...

Todos los endpoints devuelven 401, incluidos los que antes funcionaban. Es lo correcto en este punto.

3.3 Paso 3 · Las claves no se guardan: se codifican

Qué vamos a hacer

Guardar los usuarios en la base con la clave codificada, y mirar los dos hashes.

Para entenderlo mejor

La oficina no guarda tu clave en un cajón. Guarda **una huella** de ella: sirve para comprobar que la sabes, y no se puede dar la vuelta.

Y hay un detalle que sorprende a todo el mundo: **la huella de la misma clave, tomada dos veces, sale distinta**. Es a propósito.

El problema

Si guardas las claves tal cual, quien consiga leer la tabla las tiene todas. Y como la gente repite contraseñas, tiene además las de su correo y su banco.

La alternativa, y por qué no

- **En claro**: nunca, por lo de arriba.
- **Un hash simple** (MD5, SHA-256): rápido de calcular — y ésa es exactamente su desgracia. Un atacante puede probar millones por segundo, y con tablas precalculadas ni siquiera prueba.
- **BCrypt**: lenta a propósito y con sal aleatoria dentro del hash. Fue el estándar durante quince años y sigue siendo mucho mejor que un hash simple. Su problema hoy no es que esté rota: es que el hardware del atacante mejoró.
- **Argon2id**, que es lo de aquí y lo que **recomienda OWASP**: lenta a propósito, con sal aleatoria dentro del hash **y con un costo en MEMORIA**. La lentitud arruina la fuerza bruta; la sal arruina las tablas precalculadas; y la memoria arruina las tarjetas gráficas.

Y esa tercera pata es la que decide, porque es la diferencia real:

BCrypt es lenta en TIEMPO. Argon2 es lenta en tiempo Y en memoria.

Una GPU tiene miles de núcleos y muy poca memoria por núcleo. Contra BCrypt puede lanzar veinte mil cálculos en paralelo; contra Argon2, que pide **16 MB por cálculo**, se queda sin RAM antes de llegar a cien.

Argon2 tiene parámetros —memoria, pasadas, paralelismo— y elegirlos mal la deja **peor** que BCrypt. Por eso no se eligen: se usa la fábrica `defaultsForSpringSecurity_v5_8()`, que trae los valores que el equipo de Spring Security mantiene al día.

Se pega

En `practica/src/main/java/cl/dgt/seguridad/config/SeguridadConfig.java`, **dentro de la clase**:

```
@Bean
PasswordEncoder codificadorDeClaves() {
    return Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8();
}
```

Lo que vas a ver

Al arrancar, la siembra imprime los dos usuarios con su hash:

```
[semilla] ana    ADMIN    $ar-
↳ gon2id$v=19$m=16384,t=2,p=1$6pRDZ7pRwU3jaaV9oNK7Ag$EtTVmBMJb4eWLzEg3NvxJqDad+X7GbuBHBpFJiTBD/A
[semilla] luis  USUARIO  $ar-
↳ gon2id$v=19$m=16384,t=2,p=1$DiTWa388g9rj7QMybwv78A$irfjVwPs8vDvX75eHbmMeeWH6WbtpD2SSyInaxrsYeU
```

Las dos claves son la misma palabra: secreta. Y los dos hashes no se parecen en nada.

Lee la estructura, que lo dice todo:

```
$argon2id$ v=19 $ m=16384,t=2,p=1 $ 6pRDZ7pRwU3jaaV9oNK7Ag $ EtTVmBMJb4eWLz...
```

El diagrama muestra la estructura del hash `$argon2id$` con las siguientes asignaciones:

- `v=19`: la variante: Argon2id
- `m=16384`: versión del algoritmo
- `t=2`: 16 MB de memoria, 2 pasadas, 1 hilo
- `p=1`: la sal
- `6pRDZ7pRwU3jaaV9oNK7Ag`: el hash

La sal y los parámetros viajan DENTRO del hash. Por eso no hace falta guardarlos aparte, y por eso subir la memoria mañana no invalida los hashes de hoy: cada uno se verifica con los suyos.

Y el `m=16384` es el número que hay que mirar: **16 MB de RAM por cada cálculo.** Eso es lo que BCrypt no tiene.

Tus hashes van a ser distintos de éstos, y distintos en cada arranque. Es la demostración: si salieran siempre iguales, la sal no estaría haciendo su trabajo.

Vas bien si...

Ves dos hashes largos, distintos entre sí, y sabes que las dos claves originales eran la misma palabra.

Si te atascas

1 · There is no PasswordEncoder mapped for the id "null"

La clave se guardó sin codificar. Es el error clásico: falta el `PasswordEncoder`, o alguien guardó la clave tal cual saltándose el codificador.

2 · El login falla aunque la clave sea correcta.

Comprueba que lo que guardaste sea el hash y no la palabra. Y que el `PasswordEncoder` que compara sea el mismo que codificó.

3.4 Paso 4 · El carnet: emitir un token

Qué vamos a hacer

Un endpoint de login que, si las credenciales son buenas, devuelve un token firmado.

Para entenderlo mejor

El carnet que te dan al entrar. Lleva tu nombre, tu categoría y **la firma de la DGT.** Cualquier puerta puede comprobar la firma sin llamar a recepción.

El problema

Sin carnet, cada puerta tendría que preguntar quién eres, y tú tendrías que dar la contraseña otra vez. Cada envío de la contraseña es una oportunidad de que se filtre.

La alternativa, y por qué no

- **Sesión en el servidor:** el servidor recuerda quién eres y te da una cookie. Es **más simple y más seguro** para una web con plantillas — la cookie la maneja el navegador. Se descarta aquí porque ata al usuario a la instancia que lo atendió: con dos réplicas detrás de un balanceador, la mitad de las peticiones no reconocen a nadie.
- **Un token firmado (JWT)**, que es lo de aquí: **no hay nada que recordar**. El token lleva dentro quién eres y qué puedes, y cualquier instancia lo valida sola.

Y una decisión más, que este lab toma y conviene entender: la firma es **simétrica**, con una clave secreta compartida. Aquí quien emite y quien verifica son **el mismo proceso**, así que no se comparte nada. En cuanto haya varios servicios verificando, hay que pasar a **firma asimétrica** — porque con la clave compartida cualquiera de ellos podría **fabricar** carnets de administrador. Eso es el Lab 14.

Se pega

El servicio que emite, en `practica/src/main/java/cl/dgt/seguridad/services/ServicioDeTokens.java`:

```
public String emitir(Authentication autenticacion) {
    Instant ahora = Instant.now();

    String roles = autenticacion.getAuthorities().stream()
        .map(a -> a.getAuthority())
        .filter(a -> a.startsWith("ROLE_"))
        .collect(Collectors.joining(" "));

    JwtClaimsSet cuerpo = JwtClaimsSet.builder()
        .issuer("lab09")
        .issuedAt(ahora)
        .expiresAt(ahora.plus(vigencia))
        .subject(autenticacion.getName())
        .claim("scope", roles)
        .build();

    return codificador.encode(JwtEncoderParameters.from(cuerpo)).getTokenValue();
}
```

Se corre

```
curl -X POST localhost:8095/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"usuario":"ana","clave":"secreta"}'
```

Lo que vas a ver

Un token largo, de unos 260 caracteres, en tres trozos separados por puntos.

Y con la clave equivocada:

```
curl -i -X POST localhost:8095/auth/login \  
  -H 'Content-Type: application/json' \  
  -d '{"usuario":"ana","clave":"equivocada"}'
```

HTTP/1.1 401

El token no está cifrado: está firmado. Cualquiera puede leer lo que lleva dentro —pega el trozo del medio en cualquier decodificador de base64 y lo verás—. Lo que nadie puede es **cambiarlo**, porque la firma dejaría de cuadrar.

Consecuencia práctica: en un token no se meten secretos. Nombre y rol, sí. Un número de cuenta, no.

Vas bien si...

Con la clave buena recibes un token de unos 260 caracteres; con la mala, un 401.

Si te atascas

1 · El login devuelve 401 con la clave correcta.

El hash guardado no corresponde a esa clave, o el PasswordEncoder no es el mismo que la codificó.

2 · The secret key ... must be at least 256 bits

La clave de firma es demasiado corta. HMAC-SHA256 pide 32 caracteres o más.

3.5 Paso 5 · 401 no es 403

Qué vamos a hacer

Poner la regla que protege la sala interna, y comprobar **las tres respuestas**: sin carnet, con carnet equivocado, y con el bueno.

Para entenderlo mejor

- **Sin carnet** □ el guardia no sabe quién eres. **401**.
- **Con carnet de visitante** □ sabe perfectamente quién eres, y esa sala no es para ti. **403**.
- **Con carnet de fiscalizador** □ pasa. **200**.

Confundir los dos primeros es el error más repetido de todo este tema, y se nota en producción: un cliente que recibe 401 vuelve a pedir credenciales; uno que recibe 403 sabe que no tiene que insistir.

El problema

Hay que decir **qué rutas exigen qué**, y decirlo en un sitio donde no se pueda olvidar.

La alternativa, y por qué no

- **@PreAuthorize sobre cada método**: la regla vive junto al código que protege, y es la única opción cuando la regla depende del objeto — «sólo el dueño del trámite». Su precio: la regla está repartida, y para saber qué protege la aplicación hay que leerla entera.

- **Las reglas en la cadena de filtros**, que es lo de aquí: **todas juntas y en orden**, se leen de un vistazo. Su límite: sólo saben de rutas, no de datos — cuando el filtro decide, todavía no ha cargado nada.

Se pega

En practica/src/main/java/cl/dgt/seguridad/config/SeguridadConfig.java, la cadena entera:

```
@Bean
SecurityFilterChain cadena(HttpSecurity http) throws Exception {
    return http
        .sessionManagement(s ->
            ↪ s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(rutas -> rutas
            .requestMatchers("/auth/login").permitAll()
            .requestMatchers("/productos/administracion").hasAuthority(
                ↪ "SCOPE_ROLE_ADMIN")
            .anyRequest().authenticated())
        .oauth2ResourceServer(oauth -> oauth.jwt(Customizer.withDefaults()))
        .build();
}
```

Dos trampas en esa línea, y las dos caen siempre:

- **El nombre de la autoridad no es el que escribiste en el token.** El token lleva "scope": "ROLE_ADMIN", y el lector de tokens de Spring le antepone SCOPE_ a todo lo que encuentra en ese claim. La autoridad acaba llamándose **SCOPE_ROLE_ADMIN**, y eso es lo que hay que pedir. Si escribes hasRole("ADMIN") —que busca ROLE_ADMIN— **no encaja nunca**, y el síntoma es un 403 a quien sí debería pasar. Hay dos capas que añaden prefijos por su cuenta; cuando no cuadran, el error no dice nada: dice 403.
- **El orden manda.** Las reglas se evalúan de arriba abajo y anyRequest() captura todo: lo que vaya **detrás** de él no se mira nunca.

Se corre

```
curl -i localhost:8095/productos/administracion

TOKEN_LUIS=$(curl -s -X POST localhost:8095/auth/login -H 'Content-Type:
↪ application/json' \
    -d '{"usuario":"luis","clave":"secreta"}' | grep -o '"token":"[^"]*' |
    ↪ cut -d'"' -f4)
curl -i -H "Authorization: Bearer $TOKEN_LUIS"
↪ localhost:8095/productos/administracion

TOKEN_ANA=$(curl -s -X POST localhost:8095/auth/login -H 'Content-Type:
↪ application/json' \
    -d '{"usuario":"ana","clave":"secreta"}' | grep -o '"token":"[^"]*' | cut
    ↪ -d'"' -f4)
curl -i -H "Authorization: Bearer $TOKEN_ANA" localhost:8095/productos/administracion
```

Lo que vas a ver

```
sin token                -> HTTP 401
con el token de luis (USUARIO) -> HTTP 403
con el token de ana (ADMIN) -> HTTP 200
```

Tres respuestas distintas al mismo endpoint, y cada una dice una cosa distinta.

Y un token inventado:

```
curl -i -H "Authorization: Bearer inventado.no.vale"
  => localhost:8095/productos/administracion
```

```
HTTP/1.1 401
```

401, no 403: un carnet falsificado no te identifica, así que el guardia está en la primera pregunta.

Vas bien si...

Los tres códigos salen: **401**, **403** y **200**, en ese orden. Si te salen 401 en los tres, el token no está llegando; si 200 en los tres, la regla no se está aplicando.

Si te atascas

1 · Ana recibe 403 y debería recibir 200.

Es la trampa del prefijo. Decodifica el trozo del medio de su token: dirá "scope":"ROLE_ADMIN". La autoridad que Spring construye a partir de ahí se llama SCOPE_ROLE_ADMIN, y es lo que hay que escribir. Con `hasRole("ADMIN")` se busca ROLE_ADMIN, que no existe.

2 · Luis recibe 200 en la sala interna.

Tu regla está **después** de `anyRequest().authenticated()`, así que nunca se evalúa. Súbela.

3 · Todo devuelve 401, incluso con token bueno.

La cabecera va como `Authorization: Bearer <token>`, con el espacio y con Bearer escrito así. Y comprueba que la variable del token no salió vacía.

4 · /auth/login también devuelve 401.

Le falta el `permitAll()`, o está detrás de `anyRequest()`.

4 Lo que aprendiste

1 · Con seguridad puesta, todo nace cerrado.

Añadir la dependencia cierra la aplicación entera. Tú abres lo que quieras abrir — y así un endpoint nuevo no puede nacer desprotegido por olvido.

2 · Una clave no se guarda: se codifica, lenta y con sal.

Por eso la misma palabra da dos hashes distintos, y por eso una tabla precalculada no sirve de nada. La lentitud de Argon2 no es un defecto: es la defensa. Y su apetito de memoria tampoco —es lo que deja fuera a las tarjetas gráficas.

3 · El token es un carnet firmado, no un secreto.

Cualquiera puede leer lo que lleva dentro; nadie puede cambiarlo. Ahí caben tu nombre y tu rol, y no cabe nada confidencial.

4 · 401 y 403 responden preguntas distintas.

401 es «no sé quién eres»; 403 es «sé quién eres y esto no es para ti». Y la autoridad que hay que pedir se llama `SCOPE_ROLE_ADMIN`, no `ROLE_ADMIN`: el prefijo lo pone el lector de tokens — la causa número uno de un 403 inexplicable.

5 Para profundizar

- **Pega el trozo del medio de tu token** en cualquier decodificador de base64 y léelo. ¿Qué hay dentro? ¿Meterías ahí un número de cuenta?
- **Cambia una letra del token** y vuelve a pedir. ¿401 o 403? ¿Por qué?
- **Reduce la vigencia del token**: pon `lab09.jwt.vigencia-segundos: 40` en el `application.yml`, reinicia, pide un token y vuelve a usarlo al cabo de un rato. Sale 401 — y no tocaste una línea de Java. Caduca a los 40 clavados — y eso es gracias a una línea de `SeguridadConfig` que pone la **tolerancia de reloj a cero**. De fábrica son 60 segundos, así que un token de 40 viviría 100. En producción esa tolerancia se deja puesta: los relojes de dos servidores nunca coinciden al segundo, y rechazar un token por medio segundo de deriva es peor que aceptarlo por medio minuto.
- **Añade `@PreAuthorize`** a un método y compara con la regla de la cadena. ¿Cuál usarías para «sólo el dueño del trámite»?
- **Ponle `hasRole("ADMIN")`** a propósito y comprueba que ana recibe 403. Es el error del prefijo, y lo vas a ver en tu equipo algún día.

6 Antes de cerrar

Párala con Ctrl+C.

```
./mvnw clean
```

Lo que te llevas:

La seguridad cierra todo y tú abres. Las claves se codifican con sal, así que la misma clave da hashes distintos. El token es un carnet firmado y legible. Y 401 no es 403.

Lo que queda pendiente, y abre el Lab 10: tu aplicación ya se defiende de quien llama. Pero **depende de otros** — y cuando el otro no contesta, tu aplicación se queda esperando y arrastra a todos sus usuarios. En el Lab 10 se mide esa espera y se corta.